Aglet Manual

Concepts · TUI Guide · CLI Reference

2026-06-13

Contents

Chapter 1

Aglet Reference — Concepts

« Home | TUI Guide | CLI Reference

1.1 How to Use This Manual

Purpose Core concepts in aglet: items, categories, views, and the rules that connect them. This document explains what things ARE; for how to operate them see the TUI Guide or the CLI Reference.

See also Home

1.2 Overview

1.2.1 About Aglet

Purpose Aglet is a personal information manager that gives you control over tasks, notes, facts, numbers, and dates. You capture information as short items, then organize them with categories and look at them through views.

Uses People use aglet to:

- Keep a GTD-style to-do list
- Track budgets and bills
- Plan and track projects
- Collect research notes
- Build a personal knowledgebase
- Track maintenance logs

How it works You type information first and structure it afterward. Aglet can also assign categories automatically when a category name appears in an item's text or note. The same items can appear in many views at once, each a different perspective on the same database.

Basic steps

1. Enter items of information.

2. Organize them into categories (by hand or automatically).
3. Display views of those items and categories.

Interfaces Aglet has two faces over one SQLite database:

- A TUI — the interactive terminal interface (run `aglet`).
- A CLI — one-shot commands for scripts and agents.

See also Items, Categories, Views, Quick start

1.2.2 Starter Workflows

Purpose Aglet has no fixed application templates. Instead, the same items can serve many purposes by organizing them into views. Here are common ways people set aglet up.

To-do list Capture tasks as items. Create a Priority category (exclusive, with High/Normal/Low children) and a Status category. Make a "Today" or "Next" view that includes the categories you want to focus on and excludes Done.

Finance Create numeric categories such as Cost or Amount. Group bills and budget lines into a view with sections per area and a per-section column total (Sum).

Projects Create a Project category with one child per project. View the same items grouped by project, with columns for status, priority, and effort, or as a Kanban board by status.

Knowledgebase Capture reference notes as items with longer notes attached. Tag them by topic and group them in a research view alongside follow-up tasks that share the same status model.

Scheduling Give items a When date. A datebook view buckets dated items into calendar ranges so upcoming work and deadlines line up.

See also Views, Numeric categories, Datebook views

1.2.3 Quick Start

Purpose Get a working database and add your first items.

The database Aglet keeps everything in one SQLite file with a `.ag` extension. Choose a path; it is created on first use. You can name the database two ways:

- `--db work.ag` on every command, or
- `AGLET_DB=work.ag` exported once in your shell.

Open the TUI

```
aglet --db work.ag
```

Running aglet with no command opens the TUI. On a new database aglet creates the reserved categories and the All Items view for you. Press `?` at any time for the in-app help panel.

Add an item

1. Press `n` to open the new-item editor.

2. Type an item, such as: `Review Work budget Friday`
3. Press `Tab` to move to the note field; add longer context.
4. Press `Ctrl-S` to save. (Enter also saves from the title field; Esc cancels.)

From the CLI

```
aglet --db work.ag add "Buy groceries" --note "Milk, eggs"
aglet --db work.ag list
```

Next Add categories (`c`), then a view (`v`) to focus the list.

See also Add an item, Add a category, Create a view, About .ag files

1.3 Core Concepts

1.3.1 Items

Purpose An item is a line or two of information that you want to keep track of in aglet. You can assign each item to any number of categories.

Examples

- Call newspapers today to respond to stories about costs.
- Fix database connection pooling.
- Meeting with whole staff every Thursday at 11:00.

Parts

- Text — the short title (up to about 350 characters).
- Note — optional longer text attached to the item.
- Dates — Entry (created), When (to happen), Done.
- Status — open or done.
- Links — dependencies and relations to other items.

Note To add more text than the title holds, attach a note. Aglet parses dates from item text when it can, and matching category names in the text or note can be assigned automatically.

See also Add an item, Notes, Categories, Mark an item done, Dependencies

1.3.2 Categories

Purpose Categories are names you use to group related items. An item can be assigned to many categories at once (multifiling).

How they work Categories are hierarchical: each category can have a parent and children. You display and change the hierarchy in the category manager.

```
Priority
  High
  Normal
  Low
```

Types Aglet has two value kinds of category:

Kind	Symbol	Contents
Tag		Boolean membership: the item is in it or not.
Numeric	`N`	Carries a decimal value per item (Cost, Qty).

In addition, categories can be marked Exclusive (only one child assignable per item) and can use automatic assignment (implicit string matching), conditions, and actions.

Reserved Every database has the reserved categories Done, When, and Entry. They cannot be modified, deleted, or used as child category names.

See also Tag categories, Numeric categories, Exclusive categories, The hierarchy, Automatic assignment, Reserved categories

1.3.3 Views

Purpose A view is a saved perspective over the same items. Each view can filter to certain categories, hide completed items, group items into sections, and show custom columns. A database can hold many views.

Examples

- A "Work Queue" view shows open work tasks by priority.
- A "Finance" view shows bills grouped by area with totals.
- A "Projects" view groups the same items by project.
- A "Scheduling" datebook view shows dated items by week.

How they work A view does not own items; it selects them. Add an item and it appears in every view whose criteria it meets. Views are saved lenses, not separate lists.

See also Standard views, Datebook views, The All Items view, Create a view, Sections

1.3.4 Sections

Purpose A section is a group within a view that collects items matching its own criteria, under a heading. Sections let one view show several lanes of related items.

Example A "Calls" section and a "Letters" section in the same view:

```
Calls
  · Call John re: DW deal
  · Call Karla for quotes

Letters
  · Send Wendy an offer
  · Answer client request
```

How they work Each section has its own include / exclude / OR criteria, and can optionally show its children as sub-groupings. Sections can be laid out stacked vertically or as horizontal lanes (a Kanban board). Each section can carry a column summary such as a total.

Filters In Normal mode, `/` scopes a search to the focused section only; `Esc` clears that section's filter.

See also Views, Add a section, Columns, Column summaries

1.3.5 Columns

Purpose A column shows a piece of data next to each item in a section, such as a numeric category value or a date.

Types

- Numeric column — shows a numeric category's value per item, and can carry a per-section summary (Sum, Avg, Min, Max).
- Date column — shows a date such as When or Entry.
- Category value — shows whether/which category applies.

In the TUI `+` adds a column, `-` removes one, `H` / `L` move it left or right. With the highlight on a column cell, `Enter` edits that value. `f` cycles a numeric column's display format; `F` cycles its summary function. `s` / `S` (or `<` / `>`) sort the section by the column.

Example A finance section with a Cost column and a Sum footer:

```
Renewals                        Cost
  · Domain renewal               12.00
  · Insurance                   480.00
                          Sum   492.00
```

See also Add a column, Column summaries, Numeric categories, Format a numeric column

1.3.6 Notes

Purpose A note lets you add longer information to an item. The title stays short; the note holds the detail.

Examples

- A meeting agenda attached to a reminder item.
- The full description of a bug attached to a task.
- Reference text for a knowledgebase entry.

How to add In the item editor, press `Tab` to reach the note field and type. To edit a long note in your external editor, press `Ctrl-G` to open `$EDITOR`. From the CLI, use `--note`, `--append-note`, or `--note-stdin` on add and edit.

Note Automatic assignment scans note text as well as item text. A category name appearing only in the note can still trigger an automatic assignment.

See also Add a note to an item, Automatic assignment, Edit an item

1.3.7 Dependencies

Purpose Dependencies are typed links between items. They let aglet track which items are waiting on others.

Link	Meaning
`depends-on`	This item needs the other item done first.
`blocks`	This item is the prerequisite of the other (the inverse of depends-on).
`related`	A non-blocking, bidirectional association.

Types

Blocked An item is "blocked" when it has at least one prerequisite (depends-on) that is not yet done. When the prerequisite is marked done, the dependent item is no longer blocked. Blocked state is computed at query time from the link graph.

How to add In the TUI, press `b` or `B` to open the link wizard. From the CLI, use `aglet link depends-on`, `link blocks`, or `link related`.

See also Create a dependency, Filter blocked items, The link wizard, Mark an item done

1.3.8 Reserved Categories

Purpose Every aglet database contains a few built-in categories with special meaning. They are created automatically and cannot be modified, deleted, or reused as child category names.

Category	Meaning
Entry	The date/time the item was created.
When	The date/time the item is to happen.
Done	The date/time the item was marked done.

The reserved categories

Note These categories do not use implicit string matching and are non-actionable. To create a workflow category that means "finished" under an exclusive Status parent, use a name such as Complete or Completed — not Done, which is reserved.

See also Categories, Mark an item done, Claim an item

1.4 Category Types

1.4.1 Tag Categories

Purpose A tag category records boolean membership: an item either has it or does not. This is the default kind of category.

Examples Work, Personal, Urgent, Bug, Frontend, Backend.

How to make

```
aglet category create "Urgent"
```

In the category manager, press `n` (sibling) or `N` (child) and type the name.

Note A tag category can be a parent of other categories, can be exclusive, and can use automatic assignment, conditions, and actions.

See also Numeric categories, Add a category, Assign a category

1.4.2 Numeric Categories

Purpose A numeric category carries a decimal value per item, instead of plain membership. The name is up to you: Cost, Miles, Qty, Effort, Amount.

How to make

```
aglet category create "Cost" --type numeric
```

The value lives on the assignment between an item and the category, so each item can have its own Cost.

Set a value

```
aglet category set-value <item> Cost 450.00
```

In the TUI, put the highlight on the numeric column cell and press `Enter`, or edit it inline in the item editor's category list. Setting a value for the first time is the usual way to assign a numeric category to an item.

Display Numeric columns can show a per-section summary (Sum, Avg, Min, Max) and can be formatted with decimals, a currency symbol, and thousands separators.

See also Set a numeric value, Format a numeric column, Columns, Column summaries

1.4.3 Exclusive Categories

Purpose An exclusive category allows an item to be assigned to only one of its children at a time. Assigning a second child replaces the first.

Example Priority (exclusive) with children High, Normal, Low. An item can be High or Normal or Low, but never two at once. Assigning Low to an item already marked High silently switches it to Low.

How to make

```
aglet category create "Priority" --exclusive
```

In the category manager, the details pane shows an Exclusive flag you can toggle.

Uses Priority, Status, Stage — anything where exactly one value should apply.

See also Categories, The hierarchy, Assign a category

1.4.4 The Category Hierarchy

Purpose Categories form a tree. A category can have a parent and any number of children, and the database can have several root categories. The hierarchy groups related categories so you can organize and filter at different levels.

Example

```
- Area
    - Backend
    - Frontend
- Priority [exclusive]
    - High
    - Normal
    - Low
```

Where You see and change the hierarchy in the category manager (`c` or `F9`). `H/J/K/L` reorder a category; `<<` and `>>` change its depth.

Note A category cannot be deleted while it still has children; reparent or delete the children first. Reparenting to the root level is done with `--root` on the CLI.

See also Organize the hierarchy, Subsumption, Discard a category

1.4.5 Subsumption

Purpose Subsumption is the rule that assigning a child category also implies its parent. If an item is in Backend, it is also treated as being in Backend's parent, Area.

Example Assigning "Backend" to an item makes it match a view that filters on "Area", because Backend is a child of Area. The parent assignment is shown with reason "Subsumption".

Why Subsumption lets you filter broadly (everything in Area) or narrowly (just Backend) from the same assignments, without tagging each item twice.

See also The hierarchy, The assignment profile, View criteria

1.5 View Types

1.5.1 Standard Views

Purpose A standard view displays any items and any categories, filtered by criteria and optionally grouped into sections. It is the general-purpose view type.

Criteria

- Include — item must have ALL of these categories (AND).
- OR-include — item must have AT LEAST ONE of these.
- Exclude — item must have NONE of these.

Layout Items can be shown one line each or multi-line, and sections can be stacked vertically or arranged as horizontal lanes for a Kanban board (toggle with `m`).

See also Views, View criteria, Datebook views, Create a view

1.5.2 Datebook Views

Purpose A datebook view buckets dated items into calendar ranges, so upcoming work, appointments, renewals, and deadlines line up by date.

Configure A datebook view has:

- A date source — When, Entry, Done, or a date category.
- A period — day, week, month, and so on.
- An interval — how many periods per bucket.
- An anchor — where the buckets start.

Browse In the TUI, `{` and `}` step buckets, `(` and `)` step the window, and `0` jumps to today. From the CLI, `aglet view datebook-browse` shifts the window with `--offset` and `--step`.

Note You create a datebook view with `aglet view create-datebook`; a standard view and a datebook view are different types and one cannot be converted to the other.

See also Create a datebook view, Browse a datebook view, Datebook keys

1.5.3 The All Items View

Purpose All Items is the built-in view that shows every item in the database with no filtering. It is created automatically and is a system view.

Access In the TUI, press `ga` to jump to it. From the CLI:

```
aglet view show "All Items"
```

Note All Items is immutable — you cannot edit or delete it — but you can clone it into a new, mutable view if you want a copy to customize. `aglet list` without `--view` uses All Items when it is present, then falls back to the first stored view.

See also Views, Clone a view, Create a view

1.6 Auto-Assignment

1.6.1 Automatic Assignment (Implicit String Matching)

Purpose Let aglet file items for you when a category name appears in an item's text or note.

How it works When implicit string matching is enabled for a category, aglet checks both the item title and the full note text. If the category name is present, the category is assigned automatically. Assigning a child also subsumes its parent. These live matches can break automatically if the text changes; manual, action, and accepted-suggestion assignments remain sticky.

Examples Adding "Refactor Backend auth module" auto-assigns Backend (and its parent Area) because "Backend" appears in the title.

Note Command examples or acceptance criteria inside a note can accidentally match categories such as Ready, CLI, or TUI. Inspect `aglet show` provenance before assuming an assignment was manual. Turning off `enable_implicit_string` evicts live matches but not older sticky derived assignments.

See also Categories, Assign a category, Profile conditions, Subsumption, Review suggestions

Chapter 2

Aglet TUI Guide

« Home | Concepts | CLI Reference

2.1 How to Use This Manual

Purpose Complete guide to the aglet terminal interface: keybindings, interactive workflows, and the view/category editors. For scripting and batch use see the CLI Reference. Core concepts are in the Concepts Reference.

See also Home

2.2 Keys and Commands

2.2.1 TUI: Normal Mode Keys

Purpose Normal mode is the main board where items are displayed. These keys work while the highlight is on an item or column.

Items

`n`	Add a new item to the focused section
`e` / `Enter`	Edit selected item (Enter adds when empty)
`a`	Assign categories to current item or selection
`d` / `D`	Toggle done on selected item(s)
`r` / `x`	Remove from view / delete selected item(s)
`b` / `B`	Open dependency link wizard (blocked-by/blocks)
`=`	Classify selected item(s) now
`p` / `i` / `o`	Toggle preview sidebar / cycle preview mode

Selection

`Space`	Toggle selection on current item
`a` / `d` / `x` / `=`	Batch assign, done, delete, or classify
`b` / `B`	Link selected items with a dependency
`Esc`	Clear selection

Navigation

`Up`/`k` `Down`/`j`	Move between items; scroll preview when focused
`Left`/`h` `Right`/`l`	Move between sections or columns
`Tab` / `S-Tab`	Next / previous section (J/K jump section)
`[` / `]`	Move item to previous / next section (or S-Up/S-Down)
`m` / `z`	Cycle lane layout / card size

Search

`/`	Search the focused section
`g/`	Search all sections
`Esc`	Clear the active section filter

Columns

`Enter`	Edit column value (on a column cell)
`+` / `-`	Add / remove a board column
`H` / `L`	Move board column left / right
`f`	Cycle numeric column format
`F`	Cycle column summary (Sum/Avg/Min/Max)
`s` / `S` or `<` / `>`	Sort section by column (ascending/descending)

Views

`v` / `V` / `F8`	Open the view picker
`,` / `.`	Previous / next view
`ga`	Jump to the All Items view

Datebook

`{` / `}`	Step previous / next date bucket
`(` / `)`	Step the browse window

`0`	Jump to today

Global

`C`	Review pending classification suggestions
`g s` / `F10`	Open Global Settings
`c` / `F9`	Open the category manager
`u`	Toggle the hide-dependent-items filter
`Ctrl-L`	Reload data from disk
`Ctrl-Z`	Undo
`Ctrl-Shift-Z`	Redo
`?`	Toggle the help panel
`q`	Quit

See also Category manager keys, View editor keys, Select multiple items

2.2.2 TUI: Category Manager Keys

Purpose The category manager is a full-screen mode for working with the category hierarchy. Open it with `c` or `F9`; press `Esc` to return.

Navigation

`Up`/`k` `Down`/`j`	Move through the category tree
`Tab`	Move focus between the tree and the details pane

Edit

`n`	Create a sibling at the selected level
`N`	Create a child of the selected category
`e` / `F2`	Edit the selected category name
`S` / `Ctrl-S`	Save category edits
`Esc`	Return to the main view

Reorder

`H` / `J` / `K` / `L`	Reorder the selected category
`<<` / `>>`	Change the category's depth (promote / demote)

Details The details pane shows flags (Exclusive, Auto-match, Actionable), the value type (Tag or Numeric), conditions, actions, and a free-form note. Workflow roles (such as the claim category) are set in Global Settings.

See also Categories, Organize the category hierarchy, Actions

2.2.3 TUI: View Editor Keys

Purpose The view editor configures a view's filter criteria, sections, columns, layout, and aliases.

Navigation

`Tab`	Move between the editor regions (such as SECTIONS and DETAILS)
`Up/k Down/j`	Move within a region

Edit Edit criteria (include / exclude / OR-include), date ranges, display mode (single-line or multi-line), section flow (vertical stacked or horizontal lanes), unmatched-item visibility, and category aliases.

Save

`S / Ctrl-S`	Save the view
`Esc`	Cancel without saving

Note Section and per-section filters are reset when you switch views or save the editor.

See also Views, Create a view, View criteria, View aliases

2.2.4 TUI: Item Editor Keys

Purpose The item editor (opened with `n` to add or `e` to edit) is a panel with a title field, a note field, and an inline category list.

Fields

`Tab / S-Tab`	Cycle focus: Title → Note → Categories → Save → Cancel
`Ctrl-S`	Save from any field
`Enter`	Save from the title field
`Esc`	Cancel

Categories

Within the inline category list:

`j / k`	Move through categories
`Space`	Toggle a tag assignment
`/`	Filter the category list
`n`	Create a new category inline

(a numeric category shows an inline editable value field)

Notes `Ctrl-G` opens `$EDITOR` to edit the title or note in your external editor.

See also Add an item, Edit an item, Add a note, Assign a category

2.2.5 TUI: Datebook Keys

Purpose A datebook view buckets dated items into calendar ranges. These keys move the visible date window.

Keys

`{`	Step to the previous date bucket
`}`	Step to the next date bucket
`(`	Step the browse window backward
`)`	Step the browse window forward
`0`	Jump to today

Note A datebook view groups items by a date source (When, Entry, Done, or a date category) over a period such as day, week, or month.

See also Datebook views, Create a datebook view, Browse a datebook view

2.3 Working with Items

2.3.1 Add an Item

Purpose Add a new item to the database. An item is a single line of free-form text, optionally with a longer note.

TUI steps

1. Press `n` to open the new-item editor (the item is added to the focused section).
2. Type the item text, such as "Review Work budget Friday".
3. Press `Tab` to move to the Note field and the inline category checklist if you want to add detail or assignments.
4. Press `Ctrl-S` to save from any field. Enter also saves from the title field; Esc cancels.

CLI steps

```
aglet add "Review Work budget Friday"
aglet add "Pay insurance" --note "Renews annually in March"
```

How it works When an item is saved, aglet scans its text and note against category names. Categories whose names appear are assigned automatically (see Automatic assignment), and recognizable dates are parsed into the When date. The CLI prints the new item id and a new_assignments count.

Note Short item-id prefixes returned by `aglet add` can be used anywhere an item id is accepted. Parse the "created " line for the id; it is not always the last line of output.

See also Items, Edit an item, Automatic assignment, Add a note, Item editor keys

2.3.2 Edit an Item

Purpose Change an item's text, note, or done state.

TUI steps

1. Select the item.
2. Press `e` (or `Enter`) to open the editor.
3. Edit the text or Tab to the note. Toggle inline category assignments if desired.
4. Press `Ctrl-S` to save, `Esc` to cancel.

CLI steps

```
aglet edit <ITEM> --text "New title"
aglet edit <ITEM> --note "Updated note text"
aglet edit <ITEM> --done            # mark done
aglet edit <ITEM> --not-done        # reopen
```

How it works Editing text re-runs automatic assignment: implicit-string matches may be added or evicted, but manual and accepted-suggestion assignments stay sticky. Press `Ctrl-G` in the TUI text or note field to open the item in `$EDITOR`.

Note On an empty section, pressing Enter starts a new item rather than editing.

See also Add an item, Add a note, Mark an item done, Automatic assignment

2.3.3 Add a Note to an Item

Purpose Attach a longer body of text to an item. Notes hold detail that does not belong in the one-line title.

TUI steps

1. Open the item with `e` (or `n` for a new item).
2. Press `Tab` to move from the title to the Note field.

3. Type freely; the note is multi-line.
4. Press `Ctrl-G` to edit the note in `$EDITOR` for longer text.
5. Press `Ctrl-S` to save.

CLI steps

```
aglet add "Plan offsite" --note "Book venue, send agenda"
aglet edit <ITEM> --note "Revised plan"
```

How it works Note text participates in automatic assignment: a category name appearing only in the note still triggers an implicit-string match. Inspect `aglet show` provenance before assuming a visible category was assigned manually.

Note In compact list output an item with a note shows a note marker; use `--verbose` or `aglet show` to read the full note.

See also Notes, Edit an item, Automatic assignment

2.3.4 Mark an Item Done

Purpose Record that an item is complete. Done is a reserved category that also drives recurrence and dependency resolution.

TUI steps Select an item (or several with `Space`) and press `d` to toggle done. `D` toggles done on the whole current selection.

CLI steps

```
aglet edit <ITEM> --done
aglet edit <ITEM> --not-done
```

How it works Completing an item assigns the reserved Done category. A done prerequisite no longer blocks items that depend on it. If the item carries a recurrence rule, completing it schedules the next occurrence (see Recurrence).

Note Done items are hidden from most views that exclude Done and from `aglet list` unless you pass `--include-done`.

See also Reserved categories, Recurrence, Dependencies

2.3.5 Recurrence

Purpose Make an item repeat on a schedule. When a recurring item is completed, aglet generates the next occurrence automatically.

How it works A recurrence rule is attached to a dated item. Completing the current occurrence (marking it Done) advances the When date to the next scheduled date and reopens the item, so recurring chores, bills, and maintenance reappear without re-entry.

Examples A monthly "Pay insurance" item set to recur reappears with next month's date each time you mark it done.

Note Recurrence works together with the reserved When and Done categories.

See also Mark an item done, Reserved categories, Datebook views

2.3.6 Delete an Item

Purpose Remove an item from the database. Deletion is logged so the item can be restored.

TUI steps Select the item(s) and press `x` to delete. (Press `r` to remove an item from the current view without deleting it from the database.)

CLI steps

```
aglet delete <ITEM>
aglet deleted              # list deletion-log entries
aglet restore <LOG_ID>    # restore by log entry id
```

How it works Deleting writes a deletion-log entry rather than erasing the item immediately. `aglet deleted` lists log entries with their ids; `aglet restore` brings an item back.

Note `r` (remove from view) and `x` (delete) are different. Remove only changes view membership; delete affects the whole database.

See also Restore a deleted item, Items, CLI item commands

2.3.7 Restore a Deleted Item

Purpose Bring back an item that was deleted, using the deletion log.

CLI steps

```
aglet deleted               # find the log entry id
aglet restore <LOG_ID>      # restore that entry
```

How it works Every delete appends an entry to the deletion log. Restoring recreates the item with its text, note, and recorded state.

Note Restore brings the item back exactly as it was deleted; the deletion log keeps a history you can recover from.

See also Delete an item, CLI item commands

2.3.8 Move an Item Between Sections

Purpose Reposition an item into a different section of the current view.

TUI steps Select the item and press `[` to move it to the previous section or `]` to move it to the next section. Shift-Up and Shift-Down do the same. Use `h`/`l` (or Left/Right) to move between sections or columns, and `Tab` / `Shift-Tab` to move focus between sections.

How it works Sections in a standard view are defined by category criteria. Moving an item between sections adjusts its category assignments so it matches the destination section's criteria.

Note In horizontal (kanban) lane layouts, moving an item between lanes is the same operation; aglet remembers the per-lane selection.

See also Sections, Add a section, Normal mode keys

2.3.9 Search Items

Purpose Find items by text in their title or note.

TUI steps Press `/` to search within the focused section, or `g/` to search across all sections. Type the query; press `Esc` to clear the active section filter.

CLI steps

```
aglet search "budget"
aglet search "budget" --not-blocked
```

How it works CLI and TUI search both route through the same matcher over item title and note text. Per-section filters in the TUI are scoped to the focused section and reset on view switch.

Note Search matches note text, so a query can match items whose title does not contain the term.

See also Items, Select multiple items, CLI filtering

2.3.10 Select Multiple Items

Purpose Operate on several items at once — assign, complete, delete, or classify them together.

TUI steps

1. Press `Space` to toggle selection on the current item; repeat to build a selection.
2. Apply a batch operation: `a` (assign categories), `d` (done), `x` (delete), `=` (classify now), or `b` / `B` (link with a dependency).
3. Press `Esc` to clear the selection.

How it works Selection lets a single command act on many items at once. Batch operations act on every selected item.

Note With no explicit selection, the same keys act on the single item under the cursor.

See also Assign a category, Mark an item done, Create a dependency, Review suggestions

2.4 Working with Categories

2.4.1 Add a Category

Purpose Create a new category — the basic filing unit. Categories can be top-level or nested under a parent.

TUI steps

1. Press `c` or `F9` to open the category manager.
2. Press `n` to create a category at the selected level, or `N` to create a child of the selected category.
3. Type the name (Work, Personal, Urgent, ...).
4. Adjust flags such as exclusive, implicit matching, and notes in the details pane.
5. Press `Ctrl-S` to save, `Esc` to return.

CLI steps

```
aglet category create "Work"
aglet category create "High" --parent Priority
aglet category create "Priority" --exclusive
aglet category create "Cost" --type numeric
```

Note The reserved names Done, When, and Entry cannot be used. For a workflow child under an exclusive Status parent, use names such as Complete or Completed, not Done.

See also Categories, Tag categories, Numeric categories, Exclusive categories, Organize the hierarchy

2.4.2 Assign a Category to an Item

Purpose Manually file an item under a category.

TUI steps

1. Select the item (or several with `Space`).
2. Press `a` to open the inline category picker.
3. Press `Space` to toggle a category's assignment without closing, or `Enter` to apply the current result and close.
4. Press `/` to filter; from the filter box Tab, Shift-Tab, Up, or Down returns focus to the narrowed list.

CLI steps

```
aglet category assign <ITEM> Work
aglet category assign <ITEM> High
```

How it works Manual assignments are durable user choices: they stay sticky even when text changes. Assigning a child also displays its parent (assigning High also shows Priority). For exclusive parents, only one child can be assigned.

Note Use the inline checklist in the item editor to assign categories while adding or editing an item.

See also Unassign a category, Automatic assignment, Exclusive categories, Set a numeric value

2.4.3 Unassign a Category

Purpose Remove a category from an item.

TUI steps Press `a` on the item, then `Space` on the assigned category to toggle it off; `Enter` applies and closes.

CLI steps

```
aglet category unassign <ITEM> Work
```

How it works Unassigning removes the explicit assignment row and reprocesses the item. If the category still matches by implicit string or profile condition, a live (non-sticky) assignment may reappear; sticky manual/action/accepted-suggestion assignments do not.

Note To stop a category from auto-matching entirely, turn off its implicit-string matching rather than repeatedly unassigning.

See also Assign a category, Automatic assignment, Discard a category

2.4.4 Review Classification Suggestions

Purpose Accept or reject category suggestions, including experimental LLM-based ones, before they are applied.

TUI steps Press `=` to classify the selected item(s) now. Press `C` to review pending classification suggestions in the suggestion-review mode, where you can accept or dismiss each one.

How it works Classification proposes categories for an item from its text and rules. Accepting a suggestion creates a sticky assignment; dismissing it does not. It runs aglet's rule-based and LLM-based suggestions on demand for review.

Note aglet has experimental support for LLM-based categorization in addition to implicit-string and profile-condition matching.

See also Automatic assignment, Profile conditions, Assign a category

2.4.5 Set a Numeric Value

Purpose Give an item a number under a numeric category — a cost, quantity, mileage, or effort estimate.

TUI steps On a numeric column cell, press `Enter` to edit the value inline. In the item editor's inline category list, an assigned numeric category shows `- [N]` with an editable value field.

CLI steps

```
aglet category set-value <ITEM> Cost 450.00
```

How it works The value is stored on the assignment edge between the item and the numeric category. A numeric column then displays the value and can be summarized per section (sum, average, min, max).

Note Editing a numeric column cell is the primary way to assign a value to an item for the first time.

See also Numeric categories, Format a numeric column, Column summaries, Add a column

2.4.6 Format a Numeric Column

Purpose Control how a numeric category's values are displayed — decimal places, currency, and thousands separators.

TUI steps On a numeric column, press `f` to cycle the column's display format.

CLI steps

```
aglet category format <CATEGORY> ...
```

How it works Formatting is a display property of the numeric category; it does not change the stored value. The same value can appear as 1450, 1,450.00, or $1,450.00 depending on format.

Note Use `F` (capital) to cycle the column's summary function; `f` sets the value format. See Column summaries.

See also Numeric categories, Set a numeric value, Column summaries

2.4.7 Organize the Category Hierarchy

Purpose Rearrange categories — reparent, promote, demote, and reorder siblings.

TUI steps In the category manager, use `H/J/K/L` to reorder and move categories, and `<< / >>` to change a category's depth level.

CLI steps

```
aglet category reparent <CATEGORY> --parent <NEW_PARENT>
aglet category reparent <CATEGORY> --root      # make top-level
aglet category rename <CATEGORY> "New Name"
```

How it works Child order matters for exclusive parents: when several derived siblings match, the earlier child in parent order wins. Manual and accepted-suggestion assignments remain durable regardless of order.

Note Workflow roles (which categories mean Ready, claim-target, etc.) are set in Global Settings, not by hierarchy position.

See also The category hierarchy, Exclusive categories, Subsumption, Global settings

2.4.8 Discard a Category

Purpose Delete a category you no longer need.

TUI steps In the category manager, select the category and delete it.

CLI steps

```
aglet category delete "Old Category"
```

How it works Deleting a category removes it and its assignments. Reserved categories (Done, When, Entry) cannot be deleted.

Note Consider turning off implicit matching instead of deleting if you only want to stop auto-assignment but keep past filings.

See also Add a category, Unassign a category, Reserved categories

2.5 Working with Views

2.5.1 Create a View

Purpose Save a lens over the item database — a filtered, sectioned, and columned presentation you can return to.

TUI steps

1. Press `v`, `V`, or `F8` to open the view picker.
2. Press `n` to create a new view and name it (Work Queue).
3. Add include/exclude criteria and any sections.
4. Press `Ctrl-S` (or `S`) to save.

CLI steps

```
aglet view create "Work Queue" --include Work --exclude Done
aglet view list
aglet view show "Work Queue"
```

How it works A view stores its criteria, sections, columns, layout, and aliases. It updates automatically as items are added or reassigned.

Note Include criteria are AND-based: `--include Work --include Urgent` matches only items with both categories.

See also Views, View criteria, Add a section, The view editor, Clone a view

2.5.2 View Criteria

Purpose Control which items a view includes.

CLI steps

```
aglet view create "My View" --include High --include Pending
aglet view edit <VIEW> ...
```

How it works Include filters are AND-based — every included category must be present. Do not use repeated includes for mutually exclusive siblings such as Pending and In Progress; use separate sections or views instead. Views also persist hide_dependent_items, which hides items that are blocked dependents.

Note Toggle the hide-dependent-items filter in the TUI with `u`.

See also Create a view, CLI filtering, Filter blocked items, Add a section

2.5.3 Add a Section to a View

Purpose Group a view's items into labelled lanes by category criteria.

CLI steps

```
aglet view section add <VIEW> ...
aglet view section update <VIEW> ...
aglet view section remove <VIEW> ...
```

TUI steps Open the view editor (from the view picker), focus the SECTIONS pane, and add or edit sections there.

How it works Each section has its own include criteria. Items that match land in that section; section flow can be vertical (stacked) or horizontal (kanban lanes). Unmatched-item visibility is a view setting.

Note Moving an item between sections adjusts its category assignments to match the destination section.

See also Sections, Add a column, Move an item between sections, The view editor

2.5.4 Add a Column to a Section

Purpose Show a numeric category's value as a column beside each item, with an optional per-section summary.

TUI steps Press `+` to add a board column and `-` to remove one. Use `H` / `L` to move a column left or right, and `Enter` on a column cell to edit a value.

CLI steps

```
aglet view column add <VIEW> ...
aglet view column update <VIEW> ...
aglet view column remove <VIEW> ...
```

How it works Columns display numeric category values. Each column can carry a summary function shown in the section footer.

Note Columns are numeric in aglet; there is no free-text column type.

See also Columns, Column summaries, Set a numeric value, Format a numeric column

2.5.5 Column Summary Functions

Purpose Aggregate a numeric column across a section — a total, average, or extreme.

TUI steps Press `F` to cycle a numeric column's summary (Sum / Avg / Min / Max). Sort a section by a column with `s` / `S` (or `<` / `>`).

CLI steps

```
aglet view set-summary <VIEW> ...
```

How it works The summary is computed over the items in the section and shown in the section footer — for example a budget total or average effort.

Note `f` cycles the value display format; `F` cycles the summary function.

See also Add a column, Numeric categories, Format a numeric column

2.5.6 Create a Datebook View

Purpose Build a view that buckets dated items into calendar ranges.

CLI steps

```
aglet view create-datebook "Scheduling" ...
```

How it works A datebook view is a distinct view type that groups items by their When date into date-interval buckets (days, weeks, etc.). It is ideal for upcoming work, appointments, renewals, and deadlines.

Note A standard view and a datebook view are different types; one cannot be converted into the other.

See also Datebook views, Browse a datebook view, Date conditions, Datebook keys

2.5.7 Browse a Datebook View

Purpose Move the visible date window of a datebook view forward and back.

TUI steps `{` and `}` step to the previous / next bucket. `(` and `)` step the window. `0` jumps to today.

CLI steps

```
aglet view datebook-browse <VIEW> ...
```

How it works Browsing shifts which date interval is shown without changing the view definition.

Note `0` (today) is the quickest way to recenter after browsing.

See also Create a datebook view, Datebook views, Datebook keys

2.5.8 The View Editor

Purpose Configure a view's filters, sections, columns, layout, aliases, and preview behavior in one screen.

TUI steps Open a view in the editor from the view picker. `Tab` moves between regions (Criteria, Datebook, Sections, Unmatched). `Enter` operates the focused row or inline input. Save with `S` or `Ctrl-S`.

How it works The editor edits mutable view properties: include/exclude criteria, date ranges, display mode (single- or multi-line), section flow (stacked or lanes), unmatched-item visibility, and category aliases. A live preview reflects changes.

Note The All Items view is immutable; clone it first to edit a copy.

See also Create a view, View criteria, Add a section, View aliases, View editor keys

2.5.9 View Aliases

Purpose Show a category under a different display name inside one view, without changing the category itself.

CLI steps

```
aglet view alias set <VIEW> <CATEGORY> "Display Name"
aglet view alias clear <VIEW> <CATEGORY>
```

How it works An alias is per-view display metadata mapping a category to a label. It affects only how the category is shown in that view.

Note Aliases do not change category identity, filters, section titles, generated subsection labels, or board column headings.

See also The view editor, Views, CLI view commands

2.5.10 Clone a View

Purpose Make an editable copy of an existing view, including the immutable All Items view.

CLI steps

```
aglet view clone "All Items" "My Items"
```

How it works Cloning copies the source view's criteria, sections, columns, and settings into a new, mutable view that you can then customize.

Note Cloning is the way to start from All Items, which cannot itself be edited.

See also Create a view, The All Items view, Discard a view

2.5.11 Discard a View

Purpose Delete a view you no longer need.

CLI steps

```
aglet view delete "Old View"
aglet view rename "Old Name" "New Name"
```

How it works Deleting a view removes the saved presentation only; the items it showed remain in the database. The All Items view cannot be deleted.

Note Deleting a view never deletes items — it only discards the lens.

See also Create a view, Clone a view, The All Items view

2.6 Working with Dependencies

2.6.1 Create a Dependency Link

Purpose Record that one item must wait for another, or relate two items.

TUI steps Press `b` or `B` on an item (or selection) to open the dependency link wizard and choose the blocked-by / blocks relationship.

CLI steps

```
aglet link depends-on <ITEM> <PREREQUISITE>
aglet link blocks <BLOCKER> <BLOCKED>
aglet link related <ITEM_A> <ITEM_B>
```

How it works depends-on and blocks are two vocabularies for the same directed relationship; related is bidirectional. An item with an unresolved depends-on prerequisite is "blocked". Done prerequisites do not block.

Note Do not create synthetic links to mean "someone is working on this" — use claim for that. Links are for real prerequisites.

See also Dependencies, Remove a link, Filter blocked items, The link wizard

2.6.2 Remove a Link

Purpose Delete a dependency or related link between two items.

CLI steps

```
aglet unlink depends-on <ITEM> <PREREQUISITE>
aglet unlink blocks <BLOCKER> <BLOCKED>
aglet unlink related <ITEM_A> <ITEM_B>
```

How it works Removing the last unresolved depends-on prerequisite unblocks the dependent item, which can make it claimable again.

Note unlink is the canonical entry point; the depends-on and blocks forms remove the same underlying relationship from either direction.

See also Create a dependency, Dependencies, Filter blocked items

2.6.3 Filter Blocked / Not-Blocked Items

Purpose Show only items that are waiting on a prerequisite, or only those that are free to start.

TUI steps Press `u` to toggle the hide-dependent-items filter for the current view.

CLI steps

```
aglet list --blocked
aglet list --not-blocked
aglet search <query> --blocked
aglet view show "<name>" --not-blocked
```

How it works blocked means the item has at least one unresolved depends-on prerequisite, computed at query time from the dependency graph. Done dependencies do not count.

Note Dependency-state filters are derived, not assignable categories; you cannot assign "blocked" to an item.

See also Dependencies, Create a dependency, The ready list, CLI filtering

2.6.4 The Link Wizard

Purpose Create dependency links interactively in the TUI.

TUI steps Press `b` or `B` on an item or a multi-item selection to open the wizard, then pick the other item and the relationship direction (blocked-by or blocks).

How it works The wizard writes the same depends-on / blocks links as the CLI. With a selection, it can link several items at once.

Note Use the wizard for prerequisites; use claim/release for "who is working on it".

See also Create a dependency, Select multiple items, CLI link commands

2.7 Settings and Indicators

2.7.1 Global Settings

Purpose Adjust application-wide preferences and workflow roles.

TUI steps Press `g s` or `F10` to open Global Settings.

How it works Global Settings control display and behavior preferences such as auto-refresh, section borders, note glyphs, and search mode, and they define workflow roles — which categories act as Ready and as the claim target used by claim/release/ready.

Note Workflow roles live here, not in the category hierarchy; reorder or rename categories without changing which one is "Ready".

See also Claim an item, The ready list, Organize the hierarchy

2.7.2 The Status and Hint Footer

Purpose Read the two-row footer at the bottom of the TUI.

How it works The footer has two rows. The top row shows transient status — the result of your last action or a brief message. The bottom row shows persistent key hints for the current mode, so the available keys change as you move between Normal view, the category manager, the view editor, and the item editor.

Note Press `?` at any time to open the full in-app help panel, which lists every key for the current context.

See also Normal mode keys, Assignment profile

2.7.3 The Item Assignment Profile

Purpose Understand the assignments and provenance shown by `aglet show`.

How it works `aglet show` prints an item's text, note, status, and its assignments. Displayed category lists include both assigned categories and their parents (assigning High also shows Priority). Provenance distinguishes manual assignments from auto-classified ones (implicit_string or other providers), so you can see why a category is present.

Note Implicit matches scan the note as well as the title, so a category can appear because of text inside the note. Check provenance before assuming a visible category was set by hand.

See also Automatic assignment, Assign a category, Categories, CLI item commands

Chapter 3

Aglet CLI Reference

« Home | Concepts | TUI Guide

3.1 How to Use This Manual

Purpose Complete reference for the aglet command-line interface. For interactive use see the TUI Guide. Core concepts are in the Concepts Reference.

See also Home

3.2 Overview

3.2.1 About the CLI

Purpose Drive aglet from the command line — useful for scripting and for LLM coding agents.

Usage

```
aglet [--db <PATH>] <COMMAND>
```

How it works Running aglet with no subcommand opens the TUI; with a subcommand it performs that action and exits. Short item-id prefixes work anywhere an item id is accepted (case-insensitive, hyphens stripped); an ambiguous prefix returns an error listing the matches. `list` and `search` default to compact one-line rows; use `--verbose` for multi-line output and `--format json` for scripts.

Examples

```
aglet --db notes.ag list
aglet --db notes.ag add "Buy groceries"
```

Note `aglet list` without `--view` uses All Items when present, then the first stored view.

See also CLI command chart, CLI item commands, CLI category commands, CLI view commands, The AGLET_DB variable

3.2.2 The AGLET_DB Environment Variable

Purpose Choose which database aglet acts on without repeating `--db`.

How it works The CLI reads the database path from `--db <path>` or, if that is absent, the `AGLET_DB` environment variable. Set `AGLET_DB` once in your shell to run a series of commands against the same file.

Examples

```
export AGLET_DB=~/notes.ag
aglet list
aglet add "Pick up parts"
```

Note `--db` on a single command overrides `AGLET_DB` for that command.

See also About .ag files, About the CLI

3.2.3 About .ag Files

Purpose Understand where aglet keeps your data.

How it works An aglet database is a single SQLite file with the `.ag` extension. It holds every item, category, view, link, and the deletion log. A new database is created automatically with the built-in categories (Done, When, Entry) and the All Items view the first time you open it.

Examples

```
cargo run --bin aglet -- --db getting-started.ag
```

Note Because the database is one file, you back it up or move it by copying that file. Schema repair runs idempotently on open.

See also The AGLET_DB variable, Reserved categories, About the CLI

3.2.4 CLI: Command Chart

Command	Purpose
`add`	Add a new item
`edit`	Edit an item's text, note, and/or done state
`show`	Show a single item with its assignments
`list`	List items (optionally filtered)
`search`	Search item text and note
`export`	Export items as Markdown
`delete`	Delete an item (writes a deletion log entry)

Command	Purpose
`deleted`	List deletion log entries
`restore`	Restore an item from the deletion log
`claim`	Atomically claim an eligible item for active work
`ready`	List items eligible to be claimed
`release`	Remove the active claim category (alias: unclaim)
`tui`	Launch the interactive TUI
`category`	Category commands (see CLI: Category commands)
`view`	View commands (see CLI: View commands)
`link`	Item-to-item link commands
`unlink`	Remove item-to-item links
`import`	Structured import commands (CSV)
`item`	Item commands in alternative noun-verb syntax

Options

`--db <PATH>`	SQLite database path (or set AGLET_DB)
`-h, --help`	Print help for any command or subcommand

Note Run `aglet` with no command to launch the TUI. Every command accepts `--help` for its own options.

See also About the CLI, Item commands, Category commands, View commands

3.3 Item Commands

3.3.1 CLI Item Commands

Purpose Create, inspect, modify, and remove items from the command line.

Commands

`add`	Add a new item (`--note`, returns the created id)
`edit`	Edit text, note, and/or done state (`--text`, `--note`, `--done`, `--not-done`)
`show`	Show a single item with its assignments
`list`	List items, optionally filtered
`search`	Search item text and note
`export`	Export items as Markdown
`delete`	Delete an item (writes deletion log)
`deleted`	List deletion-log entries
`restore`	Restore an item from the deletion log by log id

`item`	Alternative noun-verb syntax for item commands

Examples

```
aglet add "Plan offsite" --note "Book venue"
aglet show <ITEM>
aglet export > items.md
```

Note Parse the "created " line from `add` for the new id; it is not always the last line printed.

See also Add an item, Edit an item, Delete an item, CLI filtering, CLI import and export

3.4 Category Commands

3.4.1 CLI Category Commands

Purpose Manage categories, assignments, conditions, and actions from the command line.

Commands

`list`	List categories as a tree
`show`	Show details for a category
`create`	Create a category (`--parent`, `--exclusive`, `--type numeric`)
`delete`	Delete a category by name
`rename`	Rename a category
`reparent`	Reparent (`--root` makes it top-level)
`update`	Update category flags
`assign`	Assign an item to a category
`unassign`	Unassign an item from a category
`set-value`	Set a numeric value assignment
`format`	Configure numeric formatting
`add-condition`	Add a profile condition
`add-date-condition`	Add a date condition
`set-condition-mode`	Set how explicit conditions combine
`remove-condition`	Remove a condition by 1-based index
`add-action`	Add an action
`remove-action`	Remove an action by 1-based index

Examples

```
aglet category create "Priority" --exclusive
aglet category create "High" --parent Priority
aglet category assign <ITEM> High
```

```
aglet category set-value <ITEM> Cost 450.00
```

Note Done, When, and Entry are reserved and cannot be created, renamed, or deleted.

See also Add a category, Assign a category, Profile conditions, Actions, Set a numeric value

3.4.2 Profile Conditions

Purpose Assign a category based on structured rules about an item, beyond a plain name match.

CLI steps

```
aglet category add-condition <CATEGORY> ...
aglet category set-condition-mode <CATEGORY> ...
aglet category remove-condition <CATEGORY> <INDEX>
```

How it works A category can carry one or more explicit conditions. The condition mode controls how multiple conditions combine. When an item satisfies the conditions, the category is derived automatically. Derived (non-sticky) assignments can break if the item stops matching.

Note In the category manager the left pane shows a readable rule-count badge such as [2 conditions]. Conditions are re-evaluated when an item's text or dates change.

See also Automatic assignment, Date conditions, Actions, CLI category commands

3.4.3 Date Conditions

Purpose Assign a category based on an item's date — for example, to bucket items by when they are due.

CLI steps

```
aglet category add-date-condition <CATEGORY> ...
```

How it works A date condition tests an item's When date against a range or relative window. When it matches, the category is derived. Date conditions power date-range categories used to build datebook views and date-grouped sections.

Note Direct SQLite writes do not sync the reserved When assignment; use aglet/CLI logic so date conditions see the date.

See also Profile conditions, Datebook views, Reserved categories

3.4.4 Actions

Purpose Make assigning one category automatically assign or remove another.

CLI steps

```
aglet category add-action <CATEGORY> ...
aglet category remove-action <CATEGORY> <INDEX>
```

How it works An action fires when its owning category is assigned to an item. An Assign action adds another category; a Remove action removes one. Actions are event-driven: adding or editing an action does not retroactively fire for items that already have the owning category. Action-applied assignments are sticky.

Note The category manager shows an action badge such as [1 action] in the left pane.

See also Profile conditions, Automatic assignment, CLI category commands

3.5 View Commands

3.5.1 CLI View Commands

Purpose Create and edit views, sections, columns, aliases, and datebooks from the command line.

Commands

`list`	List views
`show`	Show the contents of a view
`create`	Create a view from include/exclude
`edit`	Edit mutable view properties
`clone`	Clone into a new mutable view
`rename`	Rename a view
`delete`	Delete a view by name
`section add/remove/update`	Section authoring
`column add/remove/update`	Column authoring
`alias set/clear`	Per-view category display alias
`set-summary`	Set a column summary function
`set-item-label`	Set or clear the item column label
`set-remove-from-view`	Replace the remove-from-view category set
`create-datebook`	Create a datebook (date-interval) view
`datebook-browse`	Shift a datebook view's window

Examples

```
aglet view create "Work Queue" --include Work --exclude Done
aglet view show "Work Queue"
aglet view clone "All Items" "My Items"
```

Note `--include` criteria are AND-based; use sections or separate views for mutually exclusive siblings.

See also Create a view, Add a section, Add a column, View aliases, Create a datebook view

3.6 Link Commands

3.6.1 CLI Link Commands

Purpose Create and remove item-to-item links from the command line.

Commands

`link depends-on`	ITEM depends on DEPENDS_ON_ITEM
`link blocks`	BLOCKER blocks BLOCKED
`link related`	Bidirectional related link
`unlink depends-on / blocks / related`	Remove the corresponding link (canonical entry)

Examples

```
aglet link depends-on <ITEM> <PREREQUISITE>
aglet link related <A> <B>
aglet unlink depends-on <ITEM> <PREREQUISITE>
```

Note depends-on and blocks describe the same directed relationship from opposite ends; related is symmetric.

See also Create a dependency, Remove a link, Dependencies

3.7 Filtering and Sorting

3.7.1 CLI Filtering and Sorting

Purpose Narrow and order the output of `list`, `search`, and `view show`.

Flags

`--category <C>`	Repeatable, AND semantics
`--any-category <C>`	Repeatable, OR semantics
`--exclude-category <C>`	Repeatable, NOT semantics
`--blocked / --not-blocked`	Dependency-state filters (derived)
`--value-eq <C> <V>`	Numeric value equals
`--value-in <C> <CSV>`	Numeric value in a set
`--value-max <C> <V>`	Numeric value at most
`--sort <COLUMN[:asc|desc]>`	Sort order
`--view <VIEW>`	Use a specific view
`--include-done`	Include done items
`--format <FORMAT>`	Output format (e.g. json)
`--verbose`	Multi-line human output

Examples

```
aglet list --category High --not-blocked
aglet list --any-category Work --any-category Personal
aglet list --sort Cost:desc --format json
```

Note Repeated `--category` is AND; for OR across siblings use `--any-category`. blocked/not-blocked are derived, not categories.

See also View criteria, Filter blocked items, Search items, CLI item commands

3.8 Import and Export

3.8.1 CLI Import and Export

Purpose Move items into and out of aglet.

Commands

`import`	Structured import commands (e.g. CSV)
`export`	Export items as Markdown

Examples

```
aglet export > items.md
aglet import ...
```

How it works Export writes items as Markdown to standard output. Import brings structured data in. Prefer the CLI/import path over direct SQLite writes so dates and reserved When provenance are handled correctly.

Note Direct SQLite imports must use the store datetime format `YYYY-MM-DD HH:MM:SS`; ISO strings will not load as dates.

See also CLI item commands, About the CLI, About .ag files

3.9 Workflow Commands

3.9.1 Claim an Item

Purpose Atomically take an eligible item for active work, marking it as yours.

CLI steps

```
aglet claim <ITEM>
```

How it works Claiming applies the configured claim-target category. An item is claimable only if it has the configured Ready category, does not already have the claim-target category, is

not done, and is not blocked by an unresolved depends-on prerequisite. Claimability is computed, not a link type.

Note If a claim races and fails because the item is already In Progress, pick another item rather than force-assigning.

See also The ready list, Release a claim, Filter blocked items, Global settings

3.9.2 Release a Claim

Purpose Give up an item you previously claimed.

CLI steps

```
aglet release <ITEM>
aglet unclaim <ITEM>      # alias
```

How it works Releasing removes the active claim-target category, returning the item to the pool of claimable work if it still has the Ready category.

Note release and unclaim are the same command.

See also Claim an item, The ready list

3.9.3 The Ready List

Purpose See the items that are eligible to be claimed right now.

CLI steps

```
aglet ready
```

How it works `ready` lists items that have the Ready category and are not done, not already claimed, and not blocked by unresolved dependencies. It is the recommended way to pick the next piece of work.

Note Because `ready` already excludes done, claimed, and blocked items, prefer it over scanning a full list when choosing work.

See also Claim an item, Release a claim, Filter blocked items